Lecture 3 - Attention mechanisms

AI Engineering - lecture notes

1. Why attention?

An embedding table gives every token one fixed vector, but the meaning of a word depends on its context. 'Bank' in 'river bank' and 'bank account' should end up with different representations. Recurrent networks tried to carry context forward one token at a time in a single hidden state, which made long-range dependencies hard to learn and made training sequential. Attention lets every token look at every other token directly, in parallel.

2. Attention as a soft lookup

Think of a dictionary lookup. A query is compared with keys and the matching value is returned. Attention is the soft version: the query is compared with every key, the scores are turned into weights with a softmax, and the output is the weighted average of all the values.

```
score_i  = q . k_i
weight_i = exp(score_i) / sum_j exp(score_j)
output   = sum_i weight_i * v_i
```

The weights are positive and sum to one. They are recomputed for every input sentence; the learned parameters are the projection matrices that produce queries, keys and values, not the weights themselves.

3. Where Q, K and V come from

In self-attention all three are linear projections of the same token embeddings X: Q = X W_Q, K = X W_K, V = X W_V. The query says what a token is looking for, the key says what it offers, and the value is the information it hands over. Because W_Q and W_K are different, the score from token i to token j is generally not the same as from j to i. That asymmetry is useful: a verb looking for its subject is not the same relationship as the subject looking for its verb.

Shapes: X is n x d_model, W_Q and W_K are d_model x d_k, so Q K^T is n x n. One row per query, one column per key.

4. Scaled dot-product attention

The full formula is Attention(Q, K, V) = softmax(Q K^T / sqrt(d_k) + M) V, where M is a mask (0 where attention is allowed, minus infinity where it is not).

Why divide by sqrt(d_k)? If the entries of q and k are independent with mean 0 and variance 1, then each product q_i k_i has variance 1, and the dot product adds d_k of them. So the variance of q . k is d_k and its standard deviation is sqrt(d_k). With d_k = 64 the scores are typically around +/- 8.

Large scores are bad for the softmax. It depends exponentially on score differences, so big scores push it towards a one-hot distribution: one token gets almost all the weight. A saturated softmax also has tiny gradients (the derivative p(1 - p) is close to zero when p is close to 0 or 1), so the attention weights stop learning. Dividing by sqrt(d_k) brings the variance back to 1 for any dimension and keeps the softmax in its useful range.

5. Worked example (two keys, d_k = 4)

q = [1, 0, 1, 0], k1 = [2, 1, 2, 0], k2 = [0, 1, 0, 2], v1 = [1, 0], v2 = [0, 1].
Raw scores: q . k1 = 4, q . k2 = 0. Divide by sqrt(4) = 2: scores [2, 0].
Softmax: e^2 = 7.389, so weights = [7.389 / 8.389, 1 / 8.389] = [0.881, 0.119].
Output = 0.881 * v1 + 0.119 * v2 = [0.881, 0.119].
Without scaling the weights would be [0.982, 0.018] - already much closer to one-hot, and the gradient factor p(1 - p) drops from about 0.105 to about 0.018.

6. Masks

A causal mask stops a token from attending to later tokens, which a next-token predictor must not see. Add minus infinity to those scores BEFORE the softmax so their weights become exactly zero and each row still sums to one. Padding masks hide the padding positions in a batch in the same way. Do not zero the weights after the softmax - the rows would no longer sum to one.

7. Multi-head attention

One softmax row has to compromise between all the relationships a token cares about. Multi-head attention runs h attention operations in parallel, each in its own subspace of size d_k = d_model / h, then concatenates the head outputs and mixes them with an output matrix W_O. With d_model = 512 and h = 8, each head has d_k = 64 and scales by sqrt(64) = 8. The parameter count is the same as one wide head: 4 x 512 x 512 = 1,048,576 weights for Q, K, V and O.

8. Cost

The score matrix has n x n entries per head. For n = 4,096 tokens that is 16,777,216 scores, or 32 MiB per head per layer in 16-bit floats. This quadratic growth is why long contexts are expensive. FlashAttention computes the same result in tiles without storing the full matrix.

9. Order

Self-attention on its own does not know token order: shuffle the input and the outputs are just shuffled. Positional information has to be added, either as position embeddings added to the input or by rotating queries and keys (RoPE) so that scores depend on relative distance.

Summary

- Attention = softmax-weighted average of values, weights from query-key similarity.
- Q, K, V are three learned projections of the same tokens.
- Divide scores by sqrt(d_k): dot products grow with dimension, softmax saturates, gradients vanish.
- Mask with minus infinity before the softmax.
- Multiple heads = multiple relationships at the same parameter cost.
- Cost is quadratic in sequence length.

Reading for next week: the Transformer block (residual connections, layer norm, feed-forward).